Safe and Efficient Dynamic Window Approach for Differential Mobile Robots With Stochastic Dynamics Using Deterministic Sampling

Shinya Yasuda, Taichi Kumagai, and Hiroshi Yoshida, *Member, IEEE*

***Abstract*—We propose an efficient and safe dynamic window approach (DWA) by using deterministic sampling. When the system dynamics have uncertainty, the control input includes errors, so that the DWA objective function becomes a random variable. When a random-choice algorithm with a finite number of samples is used to estimate the objective function, it may miss collisions during prediction. In this work, we approximate the end-state distribution as a one-dimensional distribution for each input candidate in advance and generate sample paths deterministically to eliminate the misses to achieve safe control. Numerical experiments have shown that this method is approximately three times as efficient as the Monte Carlo method in most indoor environments.**

***Index Terms*—Collision avoidance, planning under uncertainty, motion and path planning.**

I. Introduction

MOBILE robots have been increasingly used in a variety of situations. A typical example that has a long history is a robot transporting work objects in a factory or warehouse. Autonomous vehicles [1] on public roads are another example of mobile robot applications. An important point for mobile robot control is not only efficiency but also safety, considering that mobile robot activities have changed from their initial use in a limited designated area for robots to sharing the same space with humans.

However, robot control contains inherent errors such as floor roughness and wheel slippage [2], and thus the input speed may not always be accurately reflected in reality. Therefore, it is necessary to consider the effects of errors or uncertainty when we control the robot. One representative method is to consider a safety margin, so that even if an error occurs in control, the robot will not collide with obstacles. However, if the safety margin is set too small, safety is not ensured. If it is too large, the robot may recognize it cannot pass through a gap that it should be able to pass or select a detour path, which results in deterioration of work efficiency. To achieve both high safety and high efficiency, determining the action with taking into account the robot's error is inevitable.

In this study, we investigate how to avoid static and dynamic obstacles in a differential drive robot even when the dynamics include uncertainty. We show that the proposed method, in which deterministic sampling is performed on the basis of prior knowledge, reduces the chances of collisions compared with a random-choice algorithm that requires the same computational complexity. This study focuses on obstacle avoidance by local path planning, which is dynamic path planning performed in real time to control acceleration and avoidance actions without significant deviation from a pre-planned path. Combined with higher layers of global path planning [3], which is usually a static path planning to prepare the path beforehand such as that in graph-based methods [4], the robot can be safely controlled from the starting point to the target point.

The remainder of the paper is structured as follows. In Section II, we review related work about collision avoidance techniques by local path planning. Section III defines the dynamics considered in this work. In Section IV, we explain the details of the proposed algorithm together with mathematical preparation. In Section V, we validate the proposed method through numerical experiments. Section VI concludes our work with a brief summary and mentions future work.

II. Related Work

Much research has been conducted on methods to provide safe control, and various methods have been proposed including the dynamic window approach (DWA) [5] and the potential field method (PFM) [6]. The DWA samples the control input space and selects the best input that maximize the objective function value. The PFM is based on the variational principle and is a method to find the path that gives the lowest potential energy among the paths moving through the field on which an artificial potential field is defined on the basis of the obstacle configuration. Both methods are well known, and improvements to them are still being proposed. For example, [7] proposed a PFM with improved path smoothness. In [8], the authors proposed an improved DWA method where the coefficients of the objective function are adaptively controlled using reinforcement learning with a deep neural network to improve efficiency.

Manuscript received 24 August 2022; accepted 6 March 2023. Date of publication 15 March 2023; date of current version 23 March 2023. This letter was recommended for publication by Associate Editor A. Pierson and Editor A. Bera upon evaluation of the reviewers comments. This work was supported by the National Institute of Information and Communications Technology (NICT), Japan, under Grant 00701. *(Corresponding author: Shinya Yasuda.)*

The authors are with the Data Science Research Laboratories, NEC Corporation, Kanagawa 211-8666, Japan (e-mail: shinya-yasuda@nec.com; t_kumagai@nec.com; yoshida@nec.com).

This letter has supplementary downloadable material available at https://doi.org/10.1109/LRA.2023.3257681, provided by the authors.

Digital Object Identifier 10.1109/LRA.2023.3257681

2377-3766 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:59:23 UTC from IEEE Xplore. Restrictions apply.

Although the goal of the aforementioned methods is to avoid static obstacles, in practical applications, it is often necessary to also cope with dynamic obstacles. The concept of the reciprocal velocity obstacle (RVO) [9], [10], which is an extension of the velocity obstacle (VO) [11], is well known as a local planner to avoid moving objects. The VO is the set of velocities such that the agent in interest collides with another specific agent in finite time. The RVO determines the agent's velocity on the basis of the assumption that the opponent will also take the same evasive action, i.e., its velocity will be selected outside from the VO. This makes it difficult to apply RVO to the environment where there might be non-cooperative dynamic obstacles. Although machine learning methods have been reported to avoid such non-cooperative obstacles [12], [13], it is practically difficult to collect sufficient data to learn all dynamic obstacles in the environment, and more universal avoidance is required.

To enable the robot to avoid static and dynamic obstacles in a variety of environments, various methods that extend DWA have been proposed. For example, [14] proposed a technique to determine control input by a control policy trained in advance through deep reinforcement learning in a simulation environment instead of predefined control input candidates as performed in DWA. In [15], the control input obtained by DWA is modified by a deep Q network to improve the collision avoidance rate. In [16], the authors introduced a predictive collision avoidance method for DWA, where a specially designed objective function enables the agent to avoid the opponent agent whose velocity is estimated by the Kalman filter. This method does not utilize deep learning and only assumes that the opponent velocity is nearly constant, so that it can be easily applied in various environments and requires fewer computational resources.

However, the methods introduced so far do not take into account control errors, which may degrade safety when they are applied to a real environment. To prevent this, a number of parameters to control safety may be included in the control algorithm. For example, in [14] the authors used parameters to identify the region that had a high risk of collisions. The larger the region is considered high risk, the safer the control becomes. Although such a method of increasing robustness can be combined with various control techniques, when the high-risk region is set too large, the space available for collision avoidance becomes too small, and the robot may move along an inefficient path. Therefore, it is worthwhile to consider methods that do not require human decision making for safety by handling stochastic dynamics in a systematic manner.

To handle control errors, we can use the stochastic optimal control, in which the model of the system is described as stochastic dynamics instead of the usual deterministic dynamics. In the stochastic optimal control, control inputs are determined to optimize the expectation value of the objective function. However, it is difficult to solve general stochastic optimal control problems directly, and thus in many cases the problem is approximated analytically or an approximate solution is obtained numerically using the Monte Carlo (MC) method. Analytical approaches, which include the iterative linear-quadratic-Gaussian method [17], are not suitable for non-analytical objective functions. The MC method, also known as the sample average approximation (SAA) [18], approximates the expectation value of the objective function as the sample average obtained by repeatedly solving the stochastic differential equation numerically by e.g. the Euler-Maruyama method. This method is more versatile because it can be applied to non-analytic objective functions. However, for SAA, the required number of samples is not known in advance, and it is difficult to balance the computational cost and accuracy of the solution [19]. In other words, although the number of samples should be reduced from a practical viewpoint to perform robot control with limited power consumption, collision prediction with such a small number of samples may miss potential collisions that occur with low probability, which makes it difficult to obtain a safe control input.

III. Problem Setting

To build a mathematical model of the mobile robot with uncertainty, we make use of the stochastic differential equation, which has a diffusion term in addition to the ordinal differential equation. Assume that the robot is placed at position (x, y) in two-dimensional cartesian space and its orientation angle is θ with respect to the x-axis. Let $\mathbf{x}(t) \equiv (x(t), y(t), \theta(t))^{\mathrm{T}}$ be the state of the robot at time t. The state difference of the robot without uncertainty is given by

$$\mathrm{d}\mathbf{x} = \begin{pmatrix} \mathrm{d}x \\ \mathrm{d}y \\ \mathrm{d}\theta \end{pmatrix} = \begin{pmatrix} v\cos\theta \\ v\sin\theta \\ \omega \end{pmatrix} \mathrm{d}t, \tag{1}$$

where v and ω are the centroid velocity and the angular velocity, respectively. To add the stochastic uncertainty to the aforementioned model, we follow the procedure introduced in [20]. Let ϕ_R (ϕ_L) and ω_R (ω_L) be the cumulative spinning amount and the spinning speed of the right (left) wheel, respectively. Then the stochastic effect is written as

$$\mathrm{d}\phi_i = \omega_i(t)\mathrm{d}t + \sqrt{D}\mathrm{d}w_i \tag{2}$$

for $i = R, L$, where $\mathrm{d}w_i$ is the increment of the unit-strength Wiener process and D is the diffusion constant, which affects the strength of uncertainty. Using the fact that $v\mathrm{d}t = r(\mathrm{d}\phi_R + \mathrm{d}\phi_L)/2$ and $\omega\mathrm{d}t = r(\mathrm{d}\phi_R - \mathrm{d}\phi_L)/\ell$, where r and ℓ are the radius of the wheel and the distance between two wheels, respectively, the stochastic model can be given by

$$\mathrm{d}\mathbf{x} = \begin{pmatrix} v\cos\theta \\ v\sin\theta \\ \omega \end{pmatrix} \mathrm{d}t + \frac{r\sqrt{D}}{2} \begin{pmatrix} \cos\theta & \cos\theta \\ \sin\theta & \sin\theta \\ 2/\ell & -2/\ell \end{pmatrix} \begin{pmatrix} \mathrm{d}w_R \\ \mathrm{d}w_L \end{pmatrix}. \tag{3}$$

Note that we denote $r\sqrt{D}$ as $\widetilde{D}$ for later convenience.

IV. Proposed Method

A. Motivation

The basic idea of this study is to estimate the DWA objective function on the basis of deterministic sampling, which enables rare events to be taken into account and reduces computational costs while preserving accuracy compared with MC sampling. In DWA, it is required that we prepare input candidates in

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:59:23 UTC from IEEE Xplore. Restrictions apply.

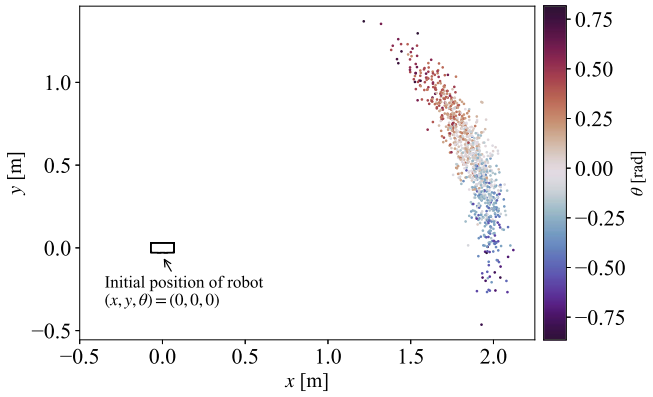


Fig. 1. Example of robot state distribution. The robot moves from its initial state $(0, 0, 0)$ in accordance with the constant control input with the noise shown in (3). The color in the plot indicates the orientation θ of the robot.

advance. In the case with the differential drive robot, they will be a set of the centroid velocity and the angular velocity. Let us denote i-th control input as c_i. For each control cycle, DWA performs simulations for all c_i and adopts c_i that gives the best objective function value as the control input. However, due to the uncertainty of the model (3), statistics on the objective function values need to be obtained for each c_i to select the control inputs. The proposed method is to compute the objective function statistics along the predetermined paths instead of MC sampling.

As a deterministic method to control stochastic systems, the unscented optimal control [21] and its variant [22], in which they utilize the unscented transform, have been proposed. The unscented transform deterministically samples a set of points called sigma points. Propagation of sigma points by the dynamics of the system gives an estimate of a mean and a covariance matrix of a new state. This procedure is known as the unscented Kalman filter [23], and it often outperforms the extended Kalman filter especially for highly nonlinear systems. However, it is assumed that the sigma points are selected from a Gaussian distribution. In fact, the state distribution produced by the model under consideration (3) is far from a Gaussian, as shown in Fig. 1.

This work is motivated by [24], where the authors show the distribution function of a robot's state can be approximated as a Gaussian on the basis of a Lie algebra. We consider the combination of the state transformation on the basis of the Lie algebra and deterministic sampling along with the dimensional reduction can balance between the computational cost and precision for DWA.

B. Brief Review of Lie Group and Lie Algebra

We briefly review the concept of the Lie group and Lie algebra. The state of the differential drive robot on the two-dimensional plane can be expressed as an element of the special Euclidean group $SE(2)$, which is an example of the Lie group. The element of $SE(2)$ consists of the in-plane rotation part and translational part, i.e.,

$$g(t_1, t_2, \vartheta) = \begin{pmatrix} \cos\vartheta & -\sin\vartheta & t_1 \\ \sin\vartheta & \cos\vartheta & t_2 \\ 0 & 0 & 1 \end{pmatrix} \in SE(2), \tag{4}$$

where $\vartheta \in \mathbb{R}$ is the parameter for the rotation and $(t_1, t_2) \in \mathbb{R}^2$ is the parameter for the translation. The Lie group formulation enables us to express the robot state $(x, y, \theta)^{\mathrm{T}}$ as an element of $SE(2)$ by substituting $t_1 = x$, $t_2 = y$, and $\vartheta = \theta$ in (4).

Next, we introduce the Lie algebra associated with $SE(2)$, referred to as $\mathfrak{se}(2)$. The Lie algebra can be defined as the set of matrix X such that $\exp(tX) \in G$ for any $t \in \mathbb{R}$ and a given Lie group G, where $\exp(\cdot)$ is the matrix exponential. The Lie algebra can be considered as the tangent space at the identity of the original Lie group. In the case with $SE(2)$, the element of $\mathfrak{se}(2)$ can be written as

$$X(v_1, v_2, \alpha) = \begin{pmatrix} 0 & -\alpha & v_1 \\ \alpha & 0 & v_2 \\ 0 & 0 & 0 \end{pmatrix}. \tag{5}$$

We also define the $\vee$ operator to map the element of $\mathfrak{se}(2)$ to the vector of parameters, i.e., $X^{\vee} = (v_1, v_2, \alpha)^{\mathrm{T}}$.

The Lie group and Lie algebra are associated with each other by exponential transformation, so that the parameters of $SE(2)$ can be expressed in terms of those of $\mathfrak{se}(2)$ and vice versa. Given that $g(t_1, t_2, \vartheta) \in SE(2)$ is associated with $X(v_1, v_2, \alpha) \in \mathfrak{se}(2)$. Calculating the matrix exponential of X yields

$$t_1 = \left[v_2(\cos\alpha - 1) + v_1 \sin\alpha\right]/\alpha, \tag{6a}$$

$$t_2 = \left[v_1(1 - \cos\alpha) + v_2 \sin\alpha\right]/\alpha, \quad \text{and} \tag{6b}$$

$$\vartheta = \alpha. \tag{6c}$$

Conversely, by taking the matrix logarithm of g,

$$X^{\vee} = \begin{pmatrix} V^{-1}\mathbf{t} \\ \vartheta \end{pmatrix} \tag{7}$$

holds, where the matrix V is defined as

$$V = \frac{1}{\vartheta} \begin{pmatrix} \sin\vartheta & \cos\vartheta - 1 \\ 1 - \cos\vartheta & \sin\vartheta \end{pmatrix} \tag{8}$$

and $\mathbf{t} = (t_1, t_2)^{\mathrm{T}}$ [25].

Let us see a physical interpretation of a state in (v_1, v_2, α)-coordinate. Considering the body velocity of the robot, which is mathematically defined as $g^{-1}\dot{g}$ for a given time-dependent rigid body motion g, gives

$$g^{-1}\dot{g} = \begin{pmatrix} 0 & -\dot{\vartheta} & \cos(\vartheta)\dot{t_1} + \sin(\vartheta)\dot{t_2} \\ \dot{\vartheta} & 0 & -\sin(\vartheta)\dot{t_1} + \cos(\vartheta)\dot{t_2} \\ 0 & 0 & 0 \end{pmatrix}, \tag{9}$$

where the dot represents the time derivative. This is nothing but the element of $\mathfrak{se}(2)$. If there is no uncertainty in the dynamics of the robot, $\dot{t_1}$ and $\dot{t_2}$ are expected to be $v\cos(\vartheta)$ and $v\sin(\vartheta)$, respectively, where v is the centroid velocity. We thus have

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:59:23 UTC from IEEE Xplore. Restrictions apply.

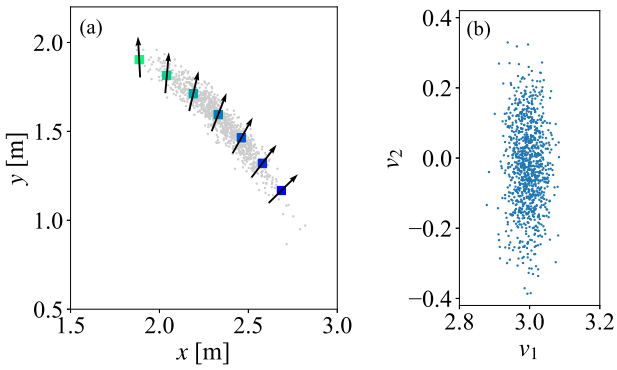


Fig. 2. (a) Example of state distribution (gray dots) and sampled states (colored squares with arrows) for input $(v, \omega) = (1.0, 0.4)$ for three seconds and $\widetilde{D} = 0.02$. The arrows denote θ at each sampled point. (b) Transformed results from gray dots shown in (a) into $\mathfrak{se}(2)$.

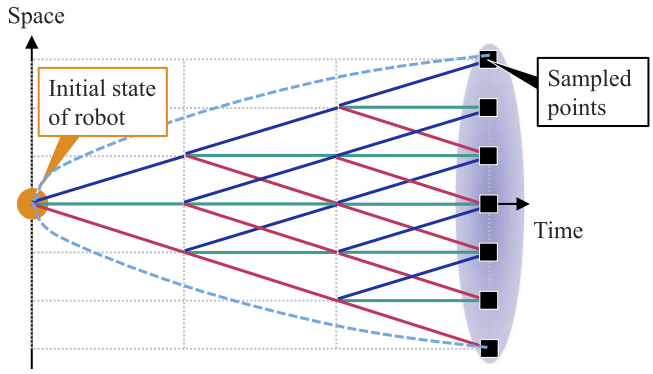


Fig. 3. Example of deterministic sample paths from the initial state (the orange circle at the leftmost side) to the sampled states (the black squares at the rightmost side). The blue ellipse shows the state distribution in (v_1, v_2, α)-space obtained from the simulations. In this example, time t_h is divided into three time steps. Each of the colored solid line represents a moving step at each time step. The black squares at rightmost are sample points. The dashed blue lines indicates the theoretical envelope of the random walk, which is proportional to the square root of time.

$(g^{-1}\dot{g})^{\vee}\Delta t = (v\Delta t, 0, \dot{\vartheta}\Delta t)^{\mathrm{T}}$, from which we can see v_1 and v_2 correspond to the moving distance along the orientation of the robot and that along the perpendicular direction, respectively [26].

C. Algorithm of Proposed Method

The proposed method is divided into two stages: (i) paths preparation and (ii) robot operation. In Stage (i), sampling paths for all control candidates are determined for a given diffusion constant. Since this stage is performed as preparation prior to robot control, it does not require real-time computation and can take sufficient computational resources. In Stage (ii), we control the robot using DWA, where the expectation value of the objective function is evaluated along the paths defined in the previous stage. Since the calculation is limited on the paths given in (i), the computational cost is light enough to perform real-time robot control. Each phase is described in detail as follows.

In Stage (i), first, (3) is numerically integrated to the prediction horizon repeatedly for all input candidates, which yields the possible states, each of which can be expressed as an element of $SE(2)$. Note that the model (3) gives the same Fokker-Planck equation by the Ito and Stratonovich methods [20], [27], so we use the Euler-Maruyama method as the numerical integration.

Next, the obtained states are rewritten in terms of the Lie algebra by transformation (7) and representative points are selected. The distribution of the rewritten states approximately becomes Gaussian in (v_1, v_2, α)-space [24]. An example of the transformation is shown in Fig. 2. The gray dots in Fig. 2(a) are transformed into the data shown in Fig. 2(b). By using the principal component analysis of the rewritten distribution, we can obtain the axis with the highest contribution. Along that axis, the points with $\pm n\sigma$ $(n \in \mathbb{N})$, where σ is the standard deviation along the axis, are selected as the representative points. The example sample points with $n = 0, \pm 1, \pm 2, \pm 3$ are shown in Fig. 2(a) as colored squares.

We then generate sample paths toward the aforementioned representative points obtained on a coarse-grained spatio-temporal grid. For the spatial dimension, we reduce the space dimension to one, which corresponds to the $n\sigma$ points along the principal component. For the temporal dimension, although it is already discretized in the unit of the control interval, the discretization is further coarsened. We consider the robot state is a random walk on the coarse-grained space-time grid and we adopt all possible walks as deterministic samples as shown in Fig. 3 as a solid colored line. For example, if space-time is discretized so that the representative points are reached in three time steps, and a walker moves $(+\sigma, 0, -\sigma)$ (each of which corresponds to the blue, green, and red solid lines in Fig. 3) at each step, the number of paths sampled is 27 since the walker can take three different actions at each time step. The probability of reaching each sample point can be described by a multinomial distribution. If we subdivide the space-time partition, this distribution will approach a Gaussian. Note that when the space-time discretization is coarse, as in the case shown in Fig. 3, the space-width of the distribution is narrower than the actual random walk, which is proportional to the square root of time, at intermediate time steps. We therefore linearly expand the width along the space axis so that at intermediate time steps the width is proportional to the square root of time.

Stage (i) is completed by obtaining the aforementioned paths for all input candidates and storing them as a table. The states between each time point is obtained by Lagrange interpolation. At this point, it is convenient to convert paths as elements of $SE(2)$ using (6).

In Stage (ii), the robot is controlled using DWA, where the objective function is expressed as a random variable. In this work, we determine the control input that optimizes the expectation value of the objective function. In other words, for all control inputs, each of which is the set of (v, ω), we estimate the expectation value of the objective function defined in the following and select the input that maximize the objective function.

In this work, we use the same objective function as proposed in [16]. The function consists of three parts: (a) whether the collision with the static obstacles occurs, (b) how early the

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:59:23 UTC from IEEE Xplore. Restrictions apply.

collision with the moving obstacles will or will not occur, and (c) how close the robot can get to the target point. Specifically, (a) is a binary function that returns whether there is an overlap between the robot and a stationary obstacle at each point on the predicted trajectory and is denoted as $a(v, \omega)$; (b), denoted as $b(v, \omega)$, is a function that returns t_k/t_h if there is a collision with a dynamic obstacle at time t_k ahead from now and 1 if there is no collision within time t_h; (c) is a function expressed as $c(v, \omega) \equiv 1 - g(v, \omega)/\max[g(v, \omega)]$, where $g(v, \omega)$ is the Euclidean distance to the target point at time t_h ahead if the control input is (v, ω). Note that to determine the value of (a), while [16] considers collisions with dilated static obstacles since the robot is regarded as a point, which yields the function value between 0 and 1, we deal strictly with the size of the robot and the static obstacles, which yields either of 0 and 1. The linear combination of these three functions is used as the objective function, namely $S(v, \omega) \equiv \alpha a(v, \omega) + \beta b(v, \omega) + \gamma c(v, \omega)$, where α, β, and γ are coefficients. For more details, please refer [16].

We conclude this section by describing the operation of the proposed method. First, we create a set of candidates of control inputs as $C = \{(v_i, \omega_j) | i, j \in \{0, 1, \ldots, N-1\}, v_i = v_0 + \delta a_{v_i} t_h, \omega_j = \omega_0 + \delta a_{\omega_j} t_h\}$, where v_0 (ω_0) is the current velocity (angular velocity) of the robot, $a_{v_i} = -A + i[2A/(N-1)]$, $a_{\omega_j} = -B + j[2B/(N-1)]$, and we choose $N = 7$, $\delta = 0.5$, $A = 0.6$, and $B = 0.4$, resulting in 49 candidates. As in [16], we also enforce the velocity limit 1.5 m/s. Next, for each candidate, we look up the table to extract the sampled paths and calculate the objective function along the paths. Taking the average of the objective function, we can obtain the expectation values of the function for each candidate. Finally, we select the candidate with the largest expectation value of the objective function as a control input.

V. Experimental Results

In this section, we show the results of the numerical experiments to demonstrate validity of the proposed method. In the experiment, we control the robot from the origin point (0,0) to the target point (10,0). At each control step, we calculate whether there is any overlap between each obstacle and the robot. If we observe the overlap, we consider the robot has collided with the obstacle, and we abort the simulation. For this reason, only one collision is observed in each trial at most. Note that even if there is no collision, the robot cannot reach the target point exactly due to the uncertainty of the dynamics, so we terminate the experiment when the x-coordinate of the robot exceeds that of the target point. In the first setup (referred to as Setup 1), we control the robot in free space with no static obstacles while avoiding dynamic obstacles coming from the side and front. The initial position of the dynamic obstacles are shown in Fig. 4 as gray rectangles. In the second setup (referred to as Setup 2), the experiments were set up to simulate the situation of an intersection of transportation routes, which is often seen in distribution warehouses. In this setup, the robot should avoid not only dynamic obstacles but also static obstacles. The setup is shown in Fig. 5. Due to the low degree of freedom of avoidance, the probability of collision should be higher than that for Setup 1.

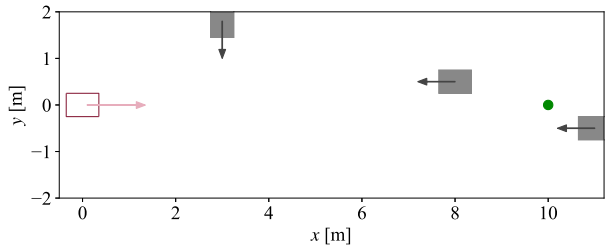


Fig. 4. Setup 1. The robot is controlled from the origin (0,0) to (10,0), which is shown as a green circle. The gray solid rectangles are moving obstacles, which move along the gray arrows.

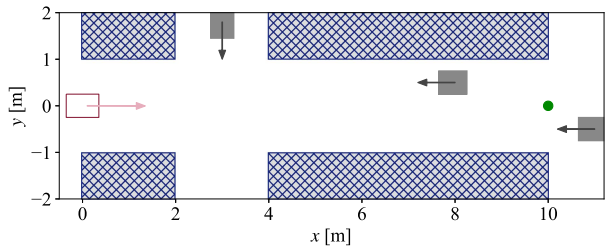


Fig. 5. Setup 2. In addition to the setup shown in Fig. 4, the robot should avoid not only dynamic obstacles but also static obstacles, which are shown as hatched rectangles.

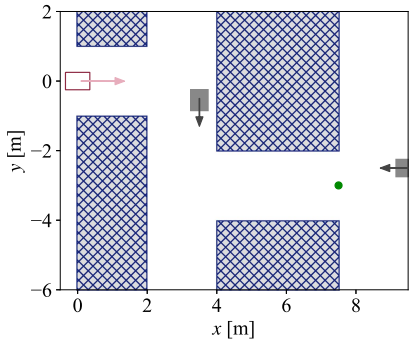


Fig. 6. Setup 3. The robot is controlled from the origin (0,0) to $(7.5, -3)$, which is shown as a green circle.

TABLE I
Parameters Used in the Experiment

Parameter(s)	Value(s)
Distance between wheels (ℓ)	0.35 m
Robot size	0.7 m (length) × 0.5 m (width)
Velocity bound	[−1.5 m/s,1.5 m/s]
Angular velocity bound	[−1 rad/s, 1 rad/s]
# of control input candidates	7 (v) × 7 (ω) = 49 (in total)
(α, β, γ)	$(0.8, 1.0, 0.5)$
Velocity of dynamic obstacles	0.75 m/s
Prediction horizon	1 sec
Control frequency	15 Hz

In the third setup (referred to as Setup 3), as shown in Fig. 6, the robot should take a more complex path than Setups 1 and 2. We sweep the diffusion constant $\widetilde{D}$ from a small value (10^{-3}) to a large value (5×10^{-2}) and obtain the collision probability. The parameters used in the experiment are summarized in Table I. In the experiment, we control the robot by the predictive collision avoidance DWA (PCA-DWA) [16], the MC method, and the

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:59:23 UTC from IEEE Xplore. Restrictions apply.

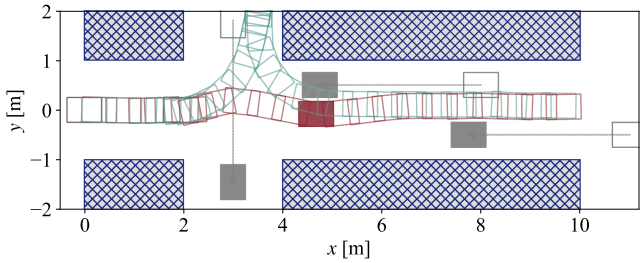


Fig. 7. Two representative contours for Setup 2 (red and green rectangles). Solid gray rectangles indicate the position of the moving obstacles at the time when the robot is at the state shown as red solid rectangles.

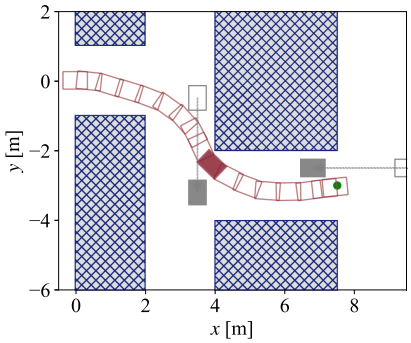


Fig. 8. Example contour for Setup 3. Solid gray rectangles indicate the position of the moving obstacles at the time when the robot is at the state shown as red solid rectangles.

proposed method. We confirm in all cases that no collisions were observed when $\widetilde{D} = 0$, i.e., no collisions occur when the model (3) has no uncertainty.

Fig. 7 shows two examples of observed contours for $\widetilde{D} = 0.02$. Most of the successful contours are similar to the red rectangles in the figure. As $\widetilde{D}$ increases, depending on the random noise, the higher number of detour paths such as the green rectangles shown in the figure are observed. In the case with the green paths, the robot waits at the left turn of the intersection until the moving obstacle that was initially in the upper right passes by. In the case with $\widetilde{D} = 0.02$, such a detour percentage becomes approximately 20%. The contour obtained for Setup 1 is similar to that shown in Fig. 7. For Setup 3, we show an example contour in Fig. 8 for $\widetilde{D} = 0.02$. We have included a supplementary video file that contains movies of the numerical experiments.

Fig. 9(a) and (c) show the observed number of collisions (left axis) and the corresponding collision probability (right axis). The number of collisions are shown on the symmetric log scale.[1] The values on the right axis are those on the left axis divided by the number of trials, 1,000. From the figures, we can see that PCA-DWA (shown as green triangles) shows higher collision probabilities than MC (blue squares) and this work (red circles) even for the small value of $\widetilde{D}$. As expected, (b) shows

[1] In the ordinary log scale, a scale is used such that $\log(x)$ is equally spaced with respect to the data x. In the symmetric log scale, $\mathrm{sgn}(x)\log(1+|x|)$ is equally spaced, where $\mathrm{sgn}(x)$ represents the sign of x. On the symmetric log scale, x can be drawn even if x is zero or negative.

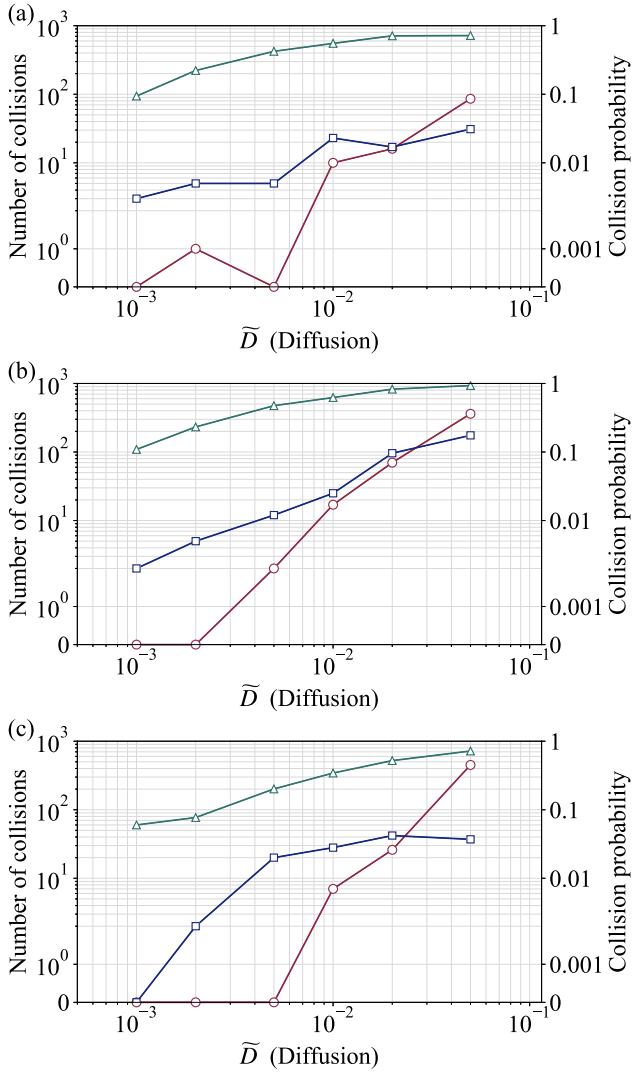


Fig. 9. Number of collisions (left axis) and corresponding collision probability (right axis) as a function of $\widetilde{D}$ for Setup 1 (a), Setup 2 (b), and Setup 3 (c). The green triangle, blue square, and red circle indicate the data of the conventional method (PCA-DWA), MC, and the proposed method, respectively. Note that the vertical axis is shown on the symmetric log scale while the horizontal axis is on the ordinary log scale.

a higher collision probability than (a), especially for large $\widetilde{D}$. It can be also seen that when the dynamics have an uncertainty, the performance of a method designed with the assumption of no errors will deteriorate significantly. MC considers the uncertainty by simulating 27 paths at each control cycle, and thus the collision probability improves by more than an order of magnitude. Compared with MC, the proposed method, which also considers 27 deterministic sampled paths, can further reduce the chance of collisions except in regions where $\widetilde{D}$ is particularly large. As $\widetilde{D}$ increases, the deterioration rate of the collision probability of this work is faster than that of MC, and the values become comparable around $\widetilde{D} \approx 0.02$ in all cases.

To further investigate the effectiveness of the proposed method, we compare the number of observed collisions and the average completion time with the data obtained by MC with a large number of sample paths. As the results with 125 and 250

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:59:23 UTC from IEEE Xplore. Restrictions apply.

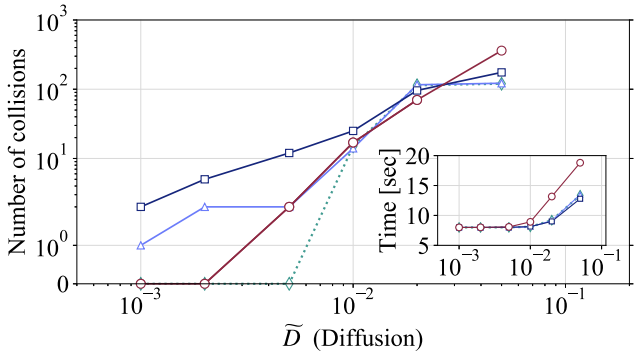


Fig. 10. Number of collisions for MC with 27 paths (blue squares), MC with 81 paths (light blue triangles), MC with 250 paths (green diamonds connected by dotted line) and the proposed method with 27 paths (red circles). The inset shows the average completion time as a function of $\widetilde{D}$.



Fig. 11. View of the floor surface used in the experiment. Part of the floor is peeled off, tilted, and uneven.

sample paths show no significant difference, we consider the MC result with 250 sample paths as the large number limit. Fig. 10 shows the number of collisions out of 1,000 trials for Setup 2 for MC with 250 sample paths (shown as green diamonds connected by a dotted line) and that for MC with 81 sample paths (light blue triangles), MC with 27 sample paths (blue squares), and the proposed method with 27 sample paths (red circles). The inset shows the average completion time over the case without any collisions. In the case with $\widetilde{D} = 10^{-3}$ and 2×10^{-3}, no collision is observed for the proposed method and MC with 250 paths, and there is no difference in the completion time. For $\widetilde{D} = 5 \times 10^{-3}$ and 10^{-2}, the number of collision is comparable with MC with 81 sample paths, while the completion time still shows little difference. However, for the data with $\widetilde{D} = 2 \times 10^{-2}$ and 5×10^{-2}, there is a large deviation in the average completion time, which means the control input determined by the proposed method becomes too conservative compared with MC and thus far from the exact solution. To summarize, these facts indicate that the proposed method with 27 sample paths shows similar or better performance than the MC with 81 sample paths for $\widetilde{D} \lesssim 10^{-2}$.

Combined with the fact that the calculation time for each control step does not differ in both methods, the proposed method is three times as efficient as MC for small $\widetilde{D}$. The actual calculation time on average is 60.7 milliseconds for MC and 58.1 milliseconds for the proposed method per control step on a computer with a Core i7-11700 processor without nontrivial parallelization. Note that the proposed method needs to create the table before the robot starts to move as explained in Section IV-C, which yields additional computational cost. However, once created, the table remains valid as long as the system dynamics, including the stochastic term, do not change. This means that the table does not need to be rebuilt each time the starting or target point changes, and the repeated use of the table enables the computational cost to create it to become relatively small. For this reason, we consider the table creation cost to be negligible in the aforementioned comparison of computational costs.

As previously shown, the proposed method can work well for small system uncertainty. This is because the state distribution can be well approximated as a one-dimensional distribution on the Lie algebra, and the use of deterministic samples has the effect of reducing the number of missed collisions that occur with low probability using existing methods. Conversely, in systems with large uncertainty, the multidimensionality of the distribution becomes larger than that in the previous case, in other words, the variance is larger in the direction of the second principal component as well. This leads to a deterioration in the approximation accuracy, and the effectiveness of the proposed method is considered to be lowered.

However, we consider the boundary value of $\widetilde{D} = 0.01$ is large enough to cover most indoor situations. Suppose the case where the robot moves at 0.6 m/s for 2 seconds. For $\widetilde{D} = 0.01$, the terminal state is distributed with a width of 236 mm (calculated by the distance between -3σ and 3σ) perpendicular to the initial robot orientation on the basis of the MC simulation. However, our experiment to move the mobile robot at 0.6 m/s for 2 seconds yields a width of 191 mm, which indicates that $\widetilde{D}$ is smaller than 0.01 even for the rough floor surface situation as shown in Fig. 11. If the floor condition is better, the error should be smaller, which leads to a smaller value of $\widetilde{D}$.

VI. Summary

In this letter, we proposed a deterministic sampling method for DWA that can be applied to a system with uncertainty. Numerical experiments have shown that the proposed method can achieve safe control by evaluating the objective function on a discretized random walk path using sample points obtained by simulation in advance, thereby eliminating missed collisions that can occur in a random-choice algorithm with a finite number of samples. In the proposed method, for the case that the uncertainty of the system is small enough, which is true for most indoor cases, the proposed method is more than three times as efficient as the conventional MC method.

References

[1] E. Yurtsever, J. Lambert, A. Carballo, and K. Takeda, "A survey of autonomous driving: Common practices and emerging technologies," *IEEE Access*, vol. 8, pp. 58443–58469, 2020.

[2] K. S. Chong and L. Kleeman, "Accurate odometry and error modelling for a mobile robot," in *Proc. IEEE Int. Conf. Robot. Automat.*, vol. 4, 1997, pp. 2783–2788.

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:59:23 UTC from IEEE Xplore. Restrictions apply.

[3] D. González, J. Pérez, V. Milanés, and F. Nashashibi, "A review of motion planning techniques for automated vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 17, no. 4, pp. 1135–1145, Apr. 2016.
[4] S. Campbell, N. O'Mahony, A. Carvalho, L. Krpalkova, D. Riordan, and J. Walsh, "Path planning techniques for mobile robots a review," in *Proc. IEEE 6th Int. Conf. Mechatron. Robot. Eng.*, 2020, pp. 12–16.
[5] D. Fox, W. Burgard, and S. Thrun, "The dynamic window approach to collision avoidance," *IEEE Robot. Automat. Mag.*, vol. 4, no. 1, pp. 23–33, Apr. 1997.
[6] E. Rimon and D. Koditschek, "Exact robot navigation using artificial potential functions," *IEEE Trans. Robot. Automat.*, vol. 8, no. 5, pp. 501–518, Oct. 1992.
[7] J. Ren, K. A. McIsaac, and R. V. Patel, "Modified Newton's method applied to potential field-based navigation for mobile robots," *IEEE Trans. Robot.*, vol. 22, no. 2, pp. 384–391, Apr. 2006.
[8] M. Dobrevski and D. Skočaj, "Adaptive dynamic window approach for local navigation," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2020, pp. 6930–6936.
[9] J. van den Berg, M. Lin, and D. Manocha, "Reciprocal velocity obstacles for real-time multi-agent navigation," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2008, pp. 1928–1935.
[10] J. Snape, J. van den Berg, S. J. Guy, and D. Manocha, "Smooth and collision-free navigation for multiple robots under differential-drive constraints," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2010, pp. 4584–4589.
[11] P. Fiorini and Z. Shiller, "Motion planning in dynamic environments using velocity obstacles," *Int. J. Robot. Res.*, vol. 17, no. 7, pp. 760–772, 1998.
[12] Y. F. Chen, M. Liu, M. Everett, and J. P. How, "Decentralized non-communicating multiagent collision avoidance with deep reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2017, pp. 285–292.
[13] M. Everett, Y. F. Chen, and J. P. How, "Motion planning among dynamic, decision-making agents with deep reinforcement learning," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2018, pp. 3052–3059.
[14] U. Patel, N. K. S. Kumar, A. J. Sathyamoorthy, and D. Manocha, "DWA-RL: Dynamically feasible deep reinforcement learning policy for robot navigation among mobile obstacles," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2021, pp. 6057–6063.
[15] J. Kim and G.-H. Yang, "Improvement of dynamic window approach using reinforcement learning in dynamic environments," *Int. J. Control, Automat. Syst.*, vol. 20, no. 9, pp. 2983–2992, 2022.
[16] M. Missura and M. Bennewitz, "Predictive collision avoidance for the dynamic window approach," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2019, pp. 8620–8626.
[17] W. Li and E. Todorov, "Iterative linearization methods for approximately optimal control and estimation of non-linear stochastic system," *Int. J. Control*, vol. 80, no. 9, pp. 1439–1453, 2007.
[18] A. Shapiro, D. Dentcheva, and A. Ruszczynski, *Lectures on Stochastic Programming: Modeling and Theory*. Philadelphia, PA, USA: SIAM, 2021.
[19] J. O. Royset, "On sample size control in sample average approximations for solving smooth stochastic programs," *Comput. Optim. Appl.*, vol. 55, no. 2, pp. 265–309, 2013.
[20] Y. Zhou and G. S. Chirikjian, "Probabilistic models of dead-reckoning error in nonholonomic mobile robots," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2003, pp. 1594–1599.
[21] I. M. Ross, R. J. Proulx, and M. Karpenko, "Unscented guidance," in *Proc. IEEE Amer. Control Conf.*, 2015, pp. 5605–5610.
[22] N. Ozaki, S. Campagnola, and R. Funase, "Tube stochastic optimal control for nonlinear constrained trajectory optimization problems," *J. Guidance, Control, Dyn.*, vol. 43, no. 4, pp. 645–655, 2020.
[23] S. J. Julier and J. K. Uhlmann, "New extension of the Kalman filter to nonlinear systems," *Proc. SPIE*, vol. 3068, pp. 182–193, 1997.
[24] A. W. Long, K. C. Wolfe, M. J. Mashner, and G. S. Chirikjian, "The banana distribution is Gaussian: A localization study with exponential coordinates," *Robotics: Sci. Syst. VIII*, pp. 265–272, 2013.
[25] E. Eade, "Lie groups for computer vision," Cambridge Univ., Cam-bridge, U.K., Tech. Rep. 2, 2014.
[26] G. S. Chirikjian, "Information theory on lie groups and mobile robotics applications," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2010, pp. 2751–2757.
[27] H. Dong and G. S. Chirikjian, "A comparison of Gaussian and Fourier methods for degenerate diffusions on SE(2)," in *Proc. IEEE 54th Conf. Decis. Control*, 2015, pp. 6016–6022.

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:59:23 UTC from IEEE Xplore. Restrictions apply.